

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 12 · JDBC y bases de datos

Conexión: DriverManager, URL JDBC y cierre de recursos

Statement frente a PreparedStatement

ResultSet: recorrido y obtención de valores

Transacciones: commit, rollback y autoCommit

Seguridad: inyección SQL y claves autogenerated

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Acceso a datos con JDBC

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Cuál es la forma habitual de cargar el driver JDBC en aplicaciones clásicas?
- a) `Class.forName("com.mysql.cj.jdbc.Driver")`
 - b) `new Driver("mysql")`
 - c) `Driver.load("mysql")`
 - d) `JDBC.register("mysql")`

RESPUESTA / SOLUCIÓN

a) `Class.forName(...)` — Carga la clase del driver, que se registra sola en `DriverManager` (en JDBC 4+ ni siquiera es necesario si el driver está en el classpath).

-
2. ¿Qué método de `DriverManager` establece la conexión con la base de datos?
- a) `DriverManager.connect(url)`
 - b) `DriverManager.getConnection(url, user, pass)`
 - c) `Connection.open(url, user, pass)`
 - d) `JDBC.connect(url, user, pass)`

RESPUESTA / SOLUCIÓN

b) `getConnection(url, user, pass)` — Devuelve un objeto `Connection` con la sesión abierta contra la BD.

-
3. ¿Qué objeto se recomienda para ejecutar consultas SQL con parámetros de forma segura?
- a) `Statement`

- b) PreparedStatement
- c) CallableStatement
- d) ResultSet

RESPUESTA / SOLUCIÓN

b) PreparedStatement — La consulta se precompila y los parámetros se envían por separado con `setString/setInt`, evitando la inyección SQL.

-
4. Usar PreparedStatement con parámetros (?) protege frente a inyección SQL porque el usuario nunca puede alterar la estructura de la consulta.

RESPUESTA / SOLUCIÓN

Verdadero — Los valores se tratan como datos, no como parte del SQL, por mucho que contengan comillas o puntos y coma.

-
5. ¿Qué método ejecuta un SELECT y devuelve las filas?
- a) `executeUpdate()`
 - b) `executeQuery()`
 - c) `executeSelect()`
 - d) `query()`

RESPUESTA / SOLUCIÓN

b) `executeQuery()` — Devuelve un `ResultSet` con las filas de la consulta.

6. ¿Qué método de `ResultSet` avanza a la siguiente fila?

- a) `next()`
- b) `hasNext()`
- c) `moveNext()`
- d) `advance()`

RESPUESTA / SOLUCIÓN

a) `next()` — Devuelve `true` si hay fila siguiente y `false` al llegar al final; se usa como condición del bucle de recorrido.

7. ¿Qué método de `ResultSet` obtiene el valor de la columna 1 como cadena?

- a) `getString(1)`
- b) `getText(1)`
- c) `getValue(1)`
- d) `read(1)`

RESPUESTA / SOLUCIÓN

a) getString(1) — Los índices de columna empiezan en 1 (no en 0).

-
8. **executeUpdate()** devuelve el número de filas afectadas por un INSERT, UPDATE o DELETE.

RESPUESTA / SOLUCIÓN

Verdadero — Para INSERT/UPDATE/DELETE se usa executeUpdate(); executeQuery() es solo para SELECT.

-
9. **¿Qué método confirma los cambios de una transacción?**

- a) conn.commit()
- b) conn.save()
- c) conn.end()
- d) conn.flush()

RESPUESTA / SOLUCIÓN

a) commit() — Hace permanentes los cambios desde el último commit/rollback.

10. ¿Cómo se obtienen las claves autogeneradas tras un INSERT (p. ej. un AUTO_INCREMENT)?

- a) `ps.getGeneratedKeys()` tras preparar con `Statement.RETURN_GENERATED_KEYS`
- b) `ps.getLastId()`
- c) `conn.getIdentity()`
- d) No es posible con JDBC

RESPUESTA / SOLUCIÓN

- a) `getGeneratedKeys()` — Se prepara la sentencia con `Statement.RETURN_GENERATED_KEYS` y se lee el `ResultSet` devuelto.
-

Parte 2: Análisis y depuración

3 puntos

1. El siguiente método tiene dos problemas graves. Encuéntralos y propón la corrección. (1,5 ptos)

```
public void buscar(String nombre) throws SQLException {  
    Connection conn = DriverManager.getConnection(url, user, pass);  
    Statement st = conn.createStatement();  
    ResultSet rs = st.executeQuery("SELECT * FROM alumnos WHERE nombre = '" + nombre + "'");  
    while (rs.next()) System.out.println(rs.getString("nombre"));  
}
```

RESPUESTA / SOLUCIÓN

- 1) Inyección SQL: el valor del usuario se concatena directamente en el SQL. Si nombre es ' OR '1'='1 se devuelven todas las filas. Debe usarse PreparedStatement con parámetro ? y setString(1, nombre).
- 2) Fuga de recursos: la conexión, el Statement y el ResultSet nunca se cierran. Debe usarse try-with-resources para cerrarlos automáticamente, incluso si hay excepción.

-
2. ¿Qué error lanza este código en tiempo de ejecución y por qué? (1,5 ptos)

```
ResultSet rs = st.executeQuery("SELECT id, nombre FROM alumnos");  
while (rs.next()) {  
    System.out.println(rs.getString(0));  
}
```

RESPUESTA / SOLUCIÓN

- Lanza SQLException: los índices de columna de ResultSet empiezan en 1, por lo que getString(0) es inválido.
- Corrección: usar getString(1) (o mejor, el nombre de la columna: rs.getString("nombre")).

Parte 3: Ejercicios de programación

6 puntos

1. Escribe un método `insertarAlumno(String nombre, int edad)` que inserte un alumno en la tabla `alumnos` (id autoincremental, nombre, edad) usando `PreparedStatement` y `try-with-resources`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.sql.*;

public class InsertarAlumno {

    public static void insertar(String nombre, int edad) {
        String url = "jdbc:mysql://localhost:3306/centro";
        String sql = "INSERT INTO alumnos (nombre, edad) VALUES (?, ?)";
        try (Connection conn = DriverManager.getConnection(url, "root", "1234");
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, nombre);
            ps.setInt(2, edad);
            ps.executeUpdate();
            System.out.println("Alumno insertado");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

2. Escribe un método `listarMayores(int edadMinima)` que muestre nombre y edad de los alumnos con edad $\geq$ `edadMinima`, ordenados por edad, usando `PreparedStatement`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.sql.*;

public class ListarAlumnos {

    public static void listarMayores(int edadMinima) {

        String url = "jdbc:mysql://localhost:3306/centro";

        String sql = "SELECT nombre, edad FROM alumnos WHERE edad >= ? ORDER BY edad";

        try (Connection conn = DriverManager.getConnection(url, "root", "1234");
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, edadMinima);

            try (ResultSet rs = ps.executeQuery()) {

                while (rs.next()) {

                    System.out.println(rs.getString("nombre") + " - " + rs.getInt("edad"));

                }

            }

        } catch (SQLException e) {

            e.printStackTrace();

        }

    }

}
```

3. Implementa una transferencia bancaria con transacción: resta el importe de la cuenta origen, lo suma a la destino y hace commit; si algo falla, rollback. Usa `setAutoCommit(false)`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.sql.*;

public class Transferencia {

    public static void transferir(int idOrigen, int idDestino, double importe) {

        String url = "jdbc:mysql://localhost:3306/banco";

        String sql = "UPDATE cuentas SET saldo = saldo + ? WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(url, "root", "1234")) {

            conn.setAutoCommit(false);

            try (PreparedStatement ps = conn.prepareStatement(sql)) {

                ps.setDouble(1, -importe); ps.setInt(2, idOrigen);

                ps.executeUpdate();

                ps.setDouble(1, importe); ps.setInt(2, idDestino);

                ps.executeUpdate();

                conn.commit();

                System.out.println("Transferencia realizada");

            } catch (SQLException e) {

                conn.rollback();

                throw e;

            }

        } catch (SQLException e) {

            e.printStackTrace();

        }

    }

}
```

Parte 4: Pregunta teórica

2 puntos

1. Explica las ventajas de PreparedStatement frente a Statement y qué es una transacción y por qué conviene usarla en operaciones de varios pasos.

RESPUESTA / SOLUCIÓN

Seguridad: los parámetros se envían separados del SQL, por lo que es inmune a la inyección SQL.

Rendimiento: la consulta se precompila una vez y se reutiliza con distintos valores (además de permitir cache de planes en muchos SGBD).

Legibilidad: el SQL queda limpio con ? en lugar de concatenaciones largas.

Transacción: conjunto de operaciones que se ejecutan como una unidad (todo o nada). Con `setAutoCommit(false)` los cambios no se persisten hasta `commit()`; si algo falla, `rollback()` deshace todo, evitando estados a medias (p. ej. dinero que sale de una cuenta y no llega a otra).

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)